

Analysis and design document

I. Architecture	1
I.1. Choix architectural : Architecture en couches	1
I.2. Justification du choix	1
II. Diagramme de classes	2
II.1. Classes métiers (/Model/)	2
II.2. Relations des classes métiers	3
II.3. Classes de service, affichage et persistance	3
II.4. Relations entre couches	3
II.5. Diagramme de classes final	4
III. Diagrammes de séquences	4
III.1. Principaux scénarios choisis	4
A. Scénario 1 – Analyser un capteur	5
B. Scénario 2 – Calculer la qualité de l'air sur une zone	5
C. Scénario 3 – Comparer des capteurs par similarité	6
D. Scénario 4 – Estimer la qualité de l'air à une position	7
E. Scénario 5 – Observer l'impact d'un purificateur d'air	8
F. Scénario 6 – Classifier la fiabilité d'un capteur privé	9
IV. Description et pseudo-code des algorithmes principaux	10
IV.1. Algorithme 1 – AdministrationService.analyserCapteur	10
IV.2. Algorithme 2 – AirQualityService.calculerMoyenneAQI	11
IV.3. Algorithme 3 – AirQualityService.comparerCapteurs	12
IV.4. Algorithme 4 – AirQualityService.estimerQualitePosition	13
IV.5. Algorithme 5 – EnvironmentalService.mesurerImpactPurificateur	14
IV.6. Algorithme 6 – AdministrationService.classifierFiabilite	15

I. Architecture

I.1. Choix architectural : Architecture en couches

Nous avons choisi une **architecture en couches** (*Layered Architecture*). Cette architecture est organisée comme suit :

- /View/ : contient les classes responsables de l'affichage et de l'interaction avec l'utilisateur via la console.
- /Service/ : contient les services métiers qui implémentent la logique applicative et les algorithmes principaux.
- /Repository/ : contient la couche d'accès aux données (lecture et écriture des fichiers CSV).
- /Model/ : contient les classes métiers représentant les entités du domaine.
- /Utils/ : contient les classes utilitaires, notamment PerformanceMonitor, qui mesure le temps d'exécution des algorithmes critiques en millisecondes.

I.2. Justification du choix

Ce choix architectural est motivé par plusieurs arguments :

Séparation des responsabilités : chaque couche a un rôle précis et unique. La vue ne contient aucune logique métier, et le repository ne connaît pas les règles de calcul. Cela rend le code plus lisible, maintenable et testable indépendamment.

Adaptée au projet : l'application AirWatcher repose sur des fichiers CSV locaux, une interface console et des algorithmes de calcul clairement identifiés. L'architecture en couches correspond naturellement à cette organisation : la couche Repository abstrait la lecture des CSV, la couche Service concentre les algorithmes (ATMO, similarité, fiabilité), et la couche View gère les menus différenciés par rôle.

Cohérence avec le diagramme de classes : le diagramme de classes réalisé reflète directement cette architecture par le regroupement en packages (Model, Repository, Service, View), ce qui garantit une correspondance directe entre la conception et l'implémentation.

Évolutivité : l'ajout de nouveaux services ou sources de données (par exemple remplacer les CSV par une base de données) se fait en modifiant uniquement la couche concernée, sans impacter les autres.

Traçabilité des performances : le PerformanceMonitor peut être injecté dans n'importe quelle méthode de service sans modifier la logique métier, conformément à l'exigence non-fonctionnelle ENF-01.

II. Diagramme de classes

II.1. Classes métiers (/Model/)

Le modèle contient les entités représentant les objets du domaine réel :

- **Utilisateur** (classe abstraite) : représente tout utilisateur du système avec un identifiant, un mot de passe, un nom et un e-mail.
- **Agence** : spécialisation d'Utilisateur pour l'agence gouvernementale, avec une référence de service et une zone de responsabilité.
- **Particulier** : spécialisation d'Utilisateur pour les citoyens contributeurs, avec un compteur de points et un indicateur de fiabilité.
- **Fournisseur** : spécialisation d'Utilisateur pour les entreprises fournissant des purificateurs.
- **Capteur** : représente un capteur physique, identifié par un ID, une position géographique (latitude, longitude) et un indicateur de fiabilité.
- **Mesure** : représente une mesure produite par un capteur à un instant donné, avec sa valeur et son statut de validité.
- **AttributMesure** : décrit le type d'une mesure (O3, SO2, NO2, PM10), son unité et sa description.
- **Purificateur** : représente un purificateur d'air avec sa position, ses dates de début et de fin de fonctionnement.
- **Rapport** : encapsule le résultat d'une analyse de capteur, avec l'état du capteur (etat : String) et la liste des anomalies détectées (anomalies : List).

- **ImpactPurificateur** : encapsule le résultat de l'évaluation d'un purificateur, avec le delta de qualité de l'air (delta : double) et le rayon d'action estimé (rayonAction : double).

II.2. Relations des classes métiers

- Un **Capteur** produit plusieurs **Mesures** (association 1 → *).
- Une **Mesure** concerne un seul **AttributMesure** (association * → 1).
- **Utilisateur** est une classe abstraite dont héritent **Fournisseur**, **Agence** et **Particulier**.
- Un **Particulier** possède zéro ou un **Capteur** (association 1 → 0..1).
- Un **Fournisseur** gère plusieurs **Purificateurs** (association 1 → *).
- **AdministrationService** retourne un **Rapport** lors de l'analyse d'un capteur.
- **EnvironmentalService** retourne un **ImpactPurificateur** lors de l'évaluation d'un purificateur.

II.3. Classes de service, affichage et persistance

- **AirQualityService (Service)** : implémente les algorithmes de calcul de l'indice ATMO, de la moyenne de qualité de l'air sur une zone, de l'estimation à une position et du classement par similarité.
- **AdministrationService (Service)** : gère l'analyse de fonctionnalité des capteurs, la classification de fiabilité des capteurs privés, et l'attribution ou la désactivation des points des particuliers.
- **EnvironmentalService (Service)** : calcule l'impact des purificateurs d'air sur leur zone d'influence (delta de qualité, rayon d'action).
- **DataReader (Repository)** : gère le chargement et l'écriture locale des fichiers CSV (sensors.csv, measurements.csv, users.csv, cleaners.csv, providers.csv). Il expose une interface unifiée d'accès aux données pour les services, incluant des méthodes de lecture (filtrage par zone, période, timestamp) ainsi que des méthodes d'écriture permettant l'ajout de nouvelles entités (addCapteur, addUtilisateur, addPurificateur, addMesure) et la mise à jour des statuts de fiabilité, conformément à l'exigence du sujet selon laquelle de nouvelles entités peuvent être ajoutées à tout moment.
- **ConsoleUI (View)** : fournit l'interface textuelle sécurisée, gère les menus différenciés par rôle (Agence, Fournisseur, Particulier) et délègue toutes les requêtes aux services appropriés.
- **PerformanceMonitor (Utils)** : outil transversal de mesure du temps d'exécution en millisecondes pour chaque algorithme majeur. Il est utilisé au début et à la fin de chaque méthode de service critique.

II.4. Relations entre couches

- **View → Service** : **ConsoleUI** délègue toutes les requêtes de calcul aux services dédiés (**AirQualityService**, **AdministrationService**, **EnvironmentalService**) via des appels de méthodes.
- **Service → Repository** : les services interrogent **DataReader** pour obtenir les données filtrées nécessaires à leurs traitements.

- **Service → Service** : certains services s'appuient sur d'autres (ex. EnvironmentalService utilise AirQualityService ; AirQualityService utilise AdministrationService pour l'attribution des points).
- **Service → Utils** : tous les services utilisent PerformanceMonitor pour mesurer leurs temps d'exécution.
- **Repository → Model** : DataReader charge les données brutes des CSV et crée les instances d'objets du modèle.

II.5. Diagramme de classes final

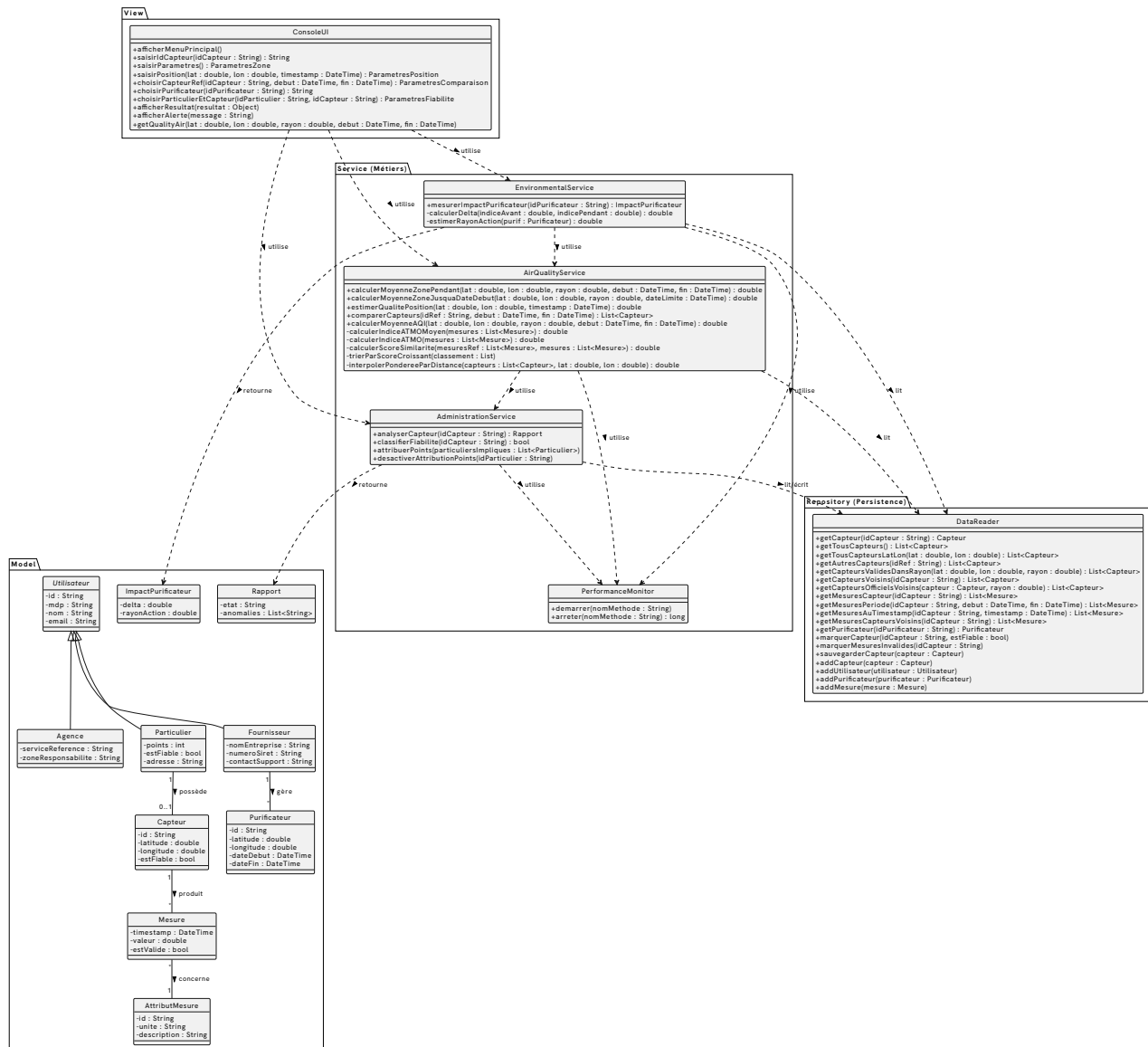


Fig. 1 : Diagramme de classes complet de l'application AirWatcher

III. Diagrammes de séquences

III.1. Principaux scénarios choisis

Les six scénarios suivant couvrent l'ensemble des fonctionnalités majeures de l'application AirWatcher. Ils illustrent les interactions entre les couches de l'architecture lors de l'exécution des cas d'utilisation principaux.

A. Scénario 1 — Analyser un capteur

Objectif : Déterminer si un capteur est fonctionnel ou défaillant en comparant ses mesures à celles de ses voisins géographiques.

Acteur : Agence.

Déroulement : L'agence saisit l'identifiant du capteur dans la console. ConsoleUI délègue à AdministrationService.analyserCapteur(). Ce service récupère via DataReader les mesures du capteur cible ainsi que la liste de ses capteurs voisins. Pour chaque voisin, il récupère également leurs mesures. Il effectue ensuite une analyse de cohérence : comparaison des moyennes par attribut (O3, SO2, NO2, PM10), détection de valeurs hors bornes physiques et de valeurs constantes suspectes. Si aucune anomalie n'est détectée, le rapport indique FONCTIONNEL ; dans le cas contraire, il indique DÉFAILLANT et liste les anomalies.

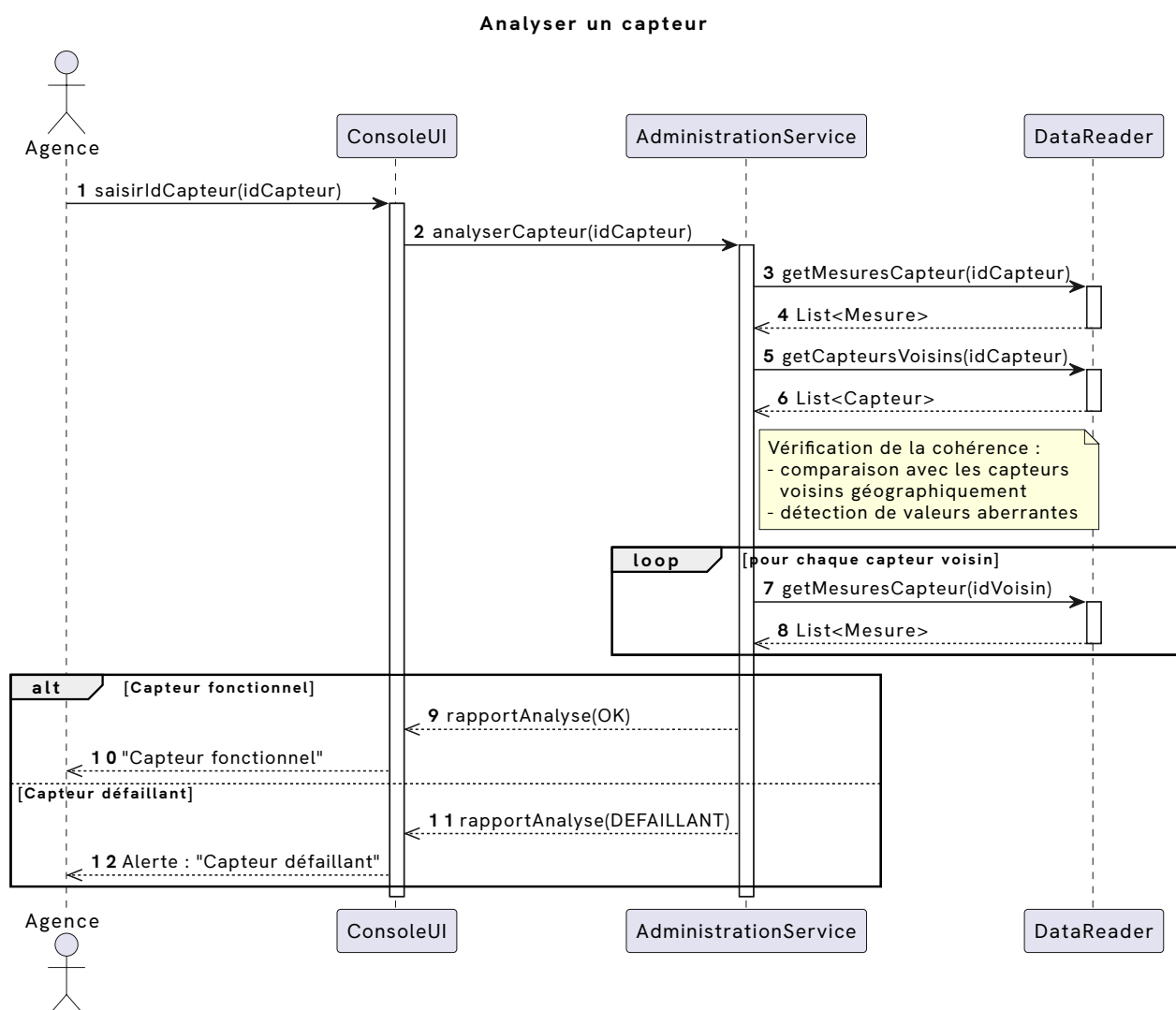


Fig. 2 : Scénario 1 — Analyser un capteur

B. Scénario 2 — Calculer la qualité de l'air sur une zone

Objectif : Calculer l'indice ATMO moyen pour une zone circulaire (définie par un centre et un rayon) sur une période donnée.

Acteur : Agence.

Déroulement : L'agence saisit les paramètres de zone (latitude, longitude, rayon, dates de début et de fin). ConsoleUI transmet ces paramètres à AirQualityService.calculerMoyenneAQI(). Le service interroge DataReader pour obtenir la liste des capteurs valides dans le rayon spécifié — les capteurs privés marqués comme non fiables sont d'emblée exclus. Pour chaque capteur retenu, les mesures valides sur la période sont récupérées. L'indice ATMO moyen est calculé à partir de l'ensemble de ces mesures. Enfin, les points de récompense sont attribués aux particuliers dont les capteurs ont contribué au calcul via AdministrationService.attribuerPoints().

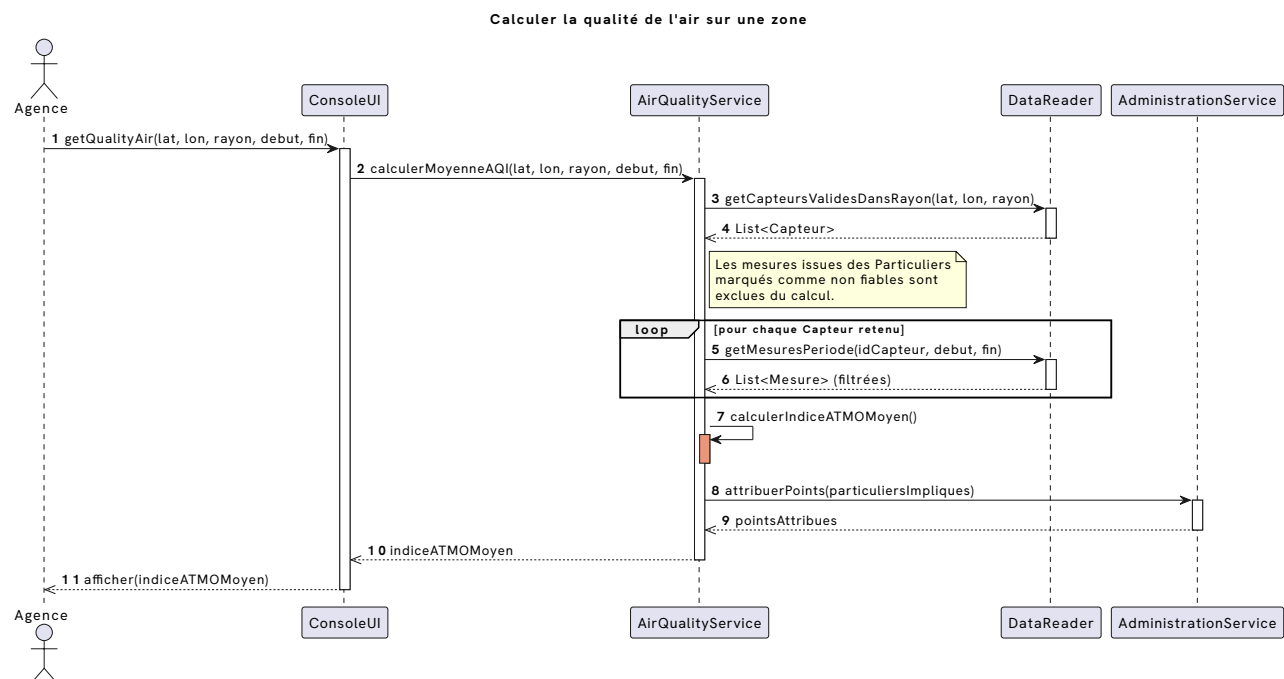


Fig. 3 : Scénario 2 — Calculer la qualité de l'air sur une zone

C. Scénario 3 — Comparer des capteurs par similarité

Objectif : Classer tous les capteurs par ordre de similarité croissante par rapport à un capteur de référence sur une période donnée.

Acteur : Agence.

Déroulement : L'agence choisit un capteur de référence et une période d'analyse. AirQualityService.comparerCapteurs() récupère d'abord les mesures du capteur de référence, puis la liste de tous les autres capteurs. Pour chaque capteur candidat, les mesures sur la même période sont récupérées, et un score de similarité est calculé. Ce score correspond à la distance euclidienne entre les vecteurs de moyennes par attribut (O3, SO2, NO2, PM10) des deux capteurs : plus le score est bas, plus le capteur est similaire à la référence. La liste est ensuite triée par score croissant et renvoyée à la console pour affichage.

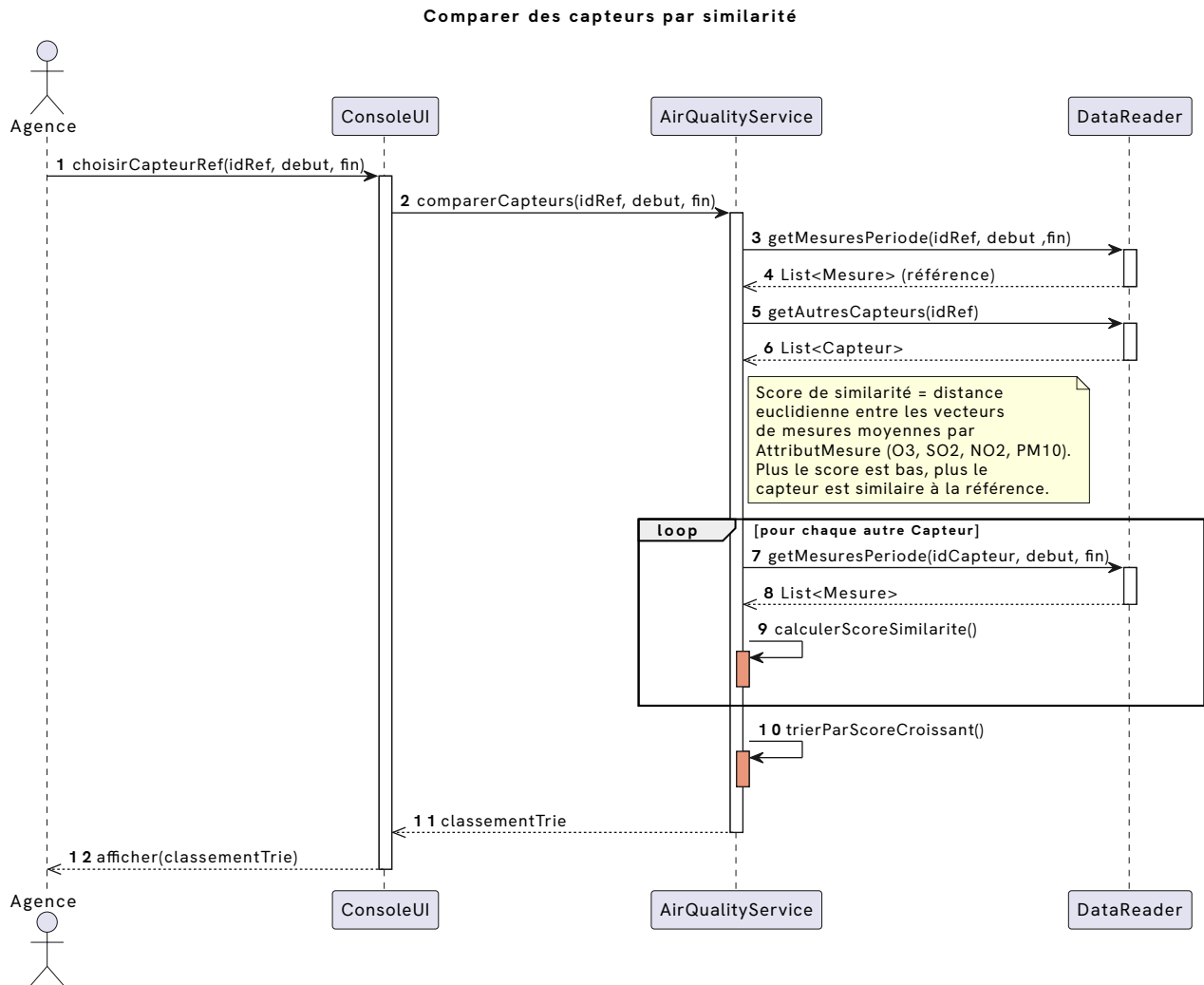


Fig. 4 : Scénario 3 — Comparer des capteurs par similarité

D. Scénario 4 — Estimer la qualité de l'air à une position

Objectif : Produire une estimation de l'indice ATMO à une position géographique précise à un instant donné, qu'un capteur soit présent ou non à cet endroit.

Acteur : Agence.

Déroulement : L'agence saisit une position (latitude, longitude) et un timestamp. `AirQualityService.estimerQualitePosition()` interroge `DataReader` pour retrouver les capteurs situés à environ 5 km du point spécifié. Si un capteur est trouvé à cette position, ses mesures au timestamp sont récupérées et l'indice ATMO est calculé directement. Dans le cas contraire, une interpolation spatiale pondérée par l'inverse de la distance (IDW) est effectuée à partir des N capteurs les plus proches. Dans les deux cas, les points de récompense sont attribués aux particuliers contributeurs.

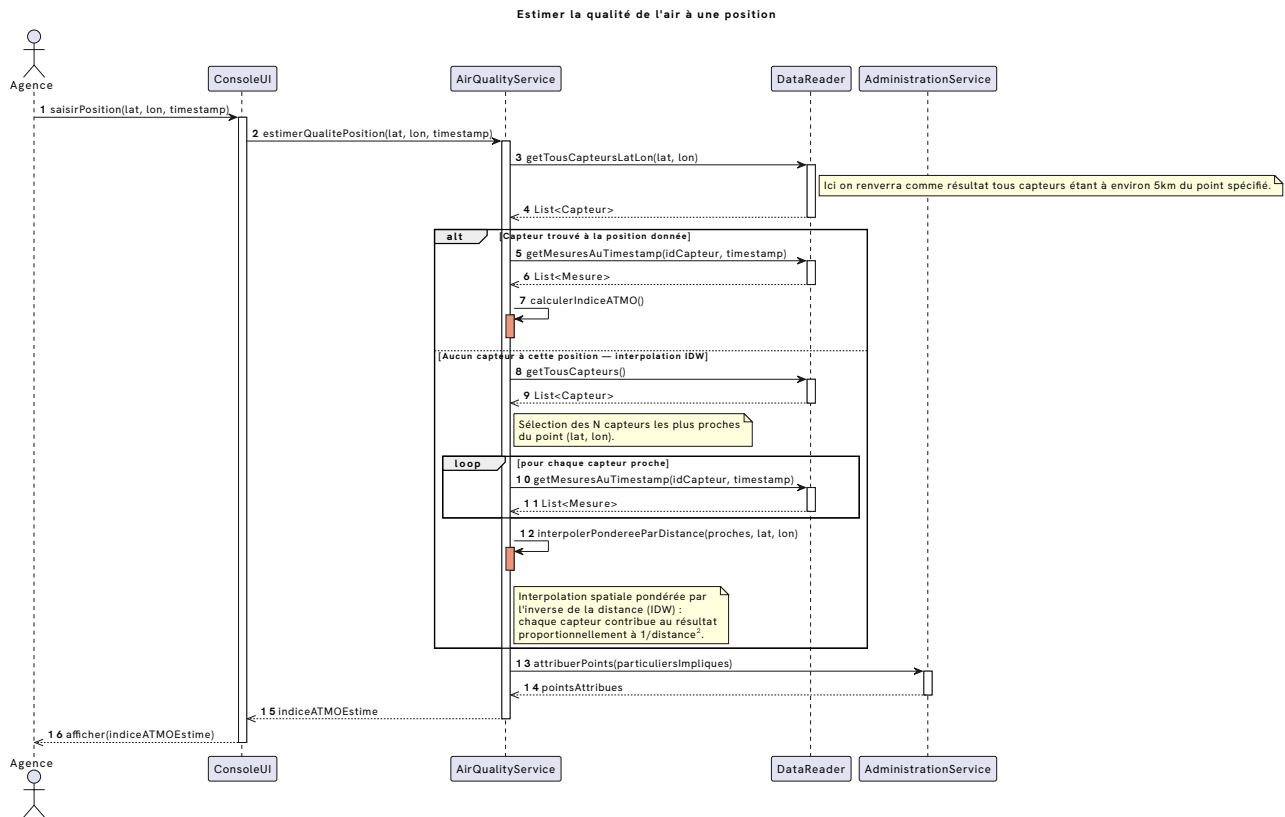


Fig. 5 : Scénario 4 — Estimer la qualité de l'air à une position

E. Scénario 5 — Observer l'impact d'un purificateur d'air

Objectif : Mesurer l'amélioration de la qualité de l'air apportée par un purificateur, en calculant le delta de l'indice ATMO avant et pendant son fonctionnement, et en estimant son rayon d'action effectif.

Acteurs : Agence ou Fournisseur.

Déroulement : L'acteur sélectionne un purificateur par son identifiant. `EnvironmentalService.mesurerImpactPurificateur()` récupère les informations du purificateur (position, dates de début et de fin). Il estime d'abord le rayon d'action en cherchant le rayon maximal autour du purificateur pour lequel une amélioration significative de la qualité de l'air est mesurable. Il calcule ensuite l'indice moyen sur la zone **avant** le démarrage du purificateur, puis l'indice **pendant** son fonctionnement, en s'appuyant sur `AirQualityService`. Le delta (différence entre les deux indices) est calculé et renvoyé avec le rayon d'action estimé.

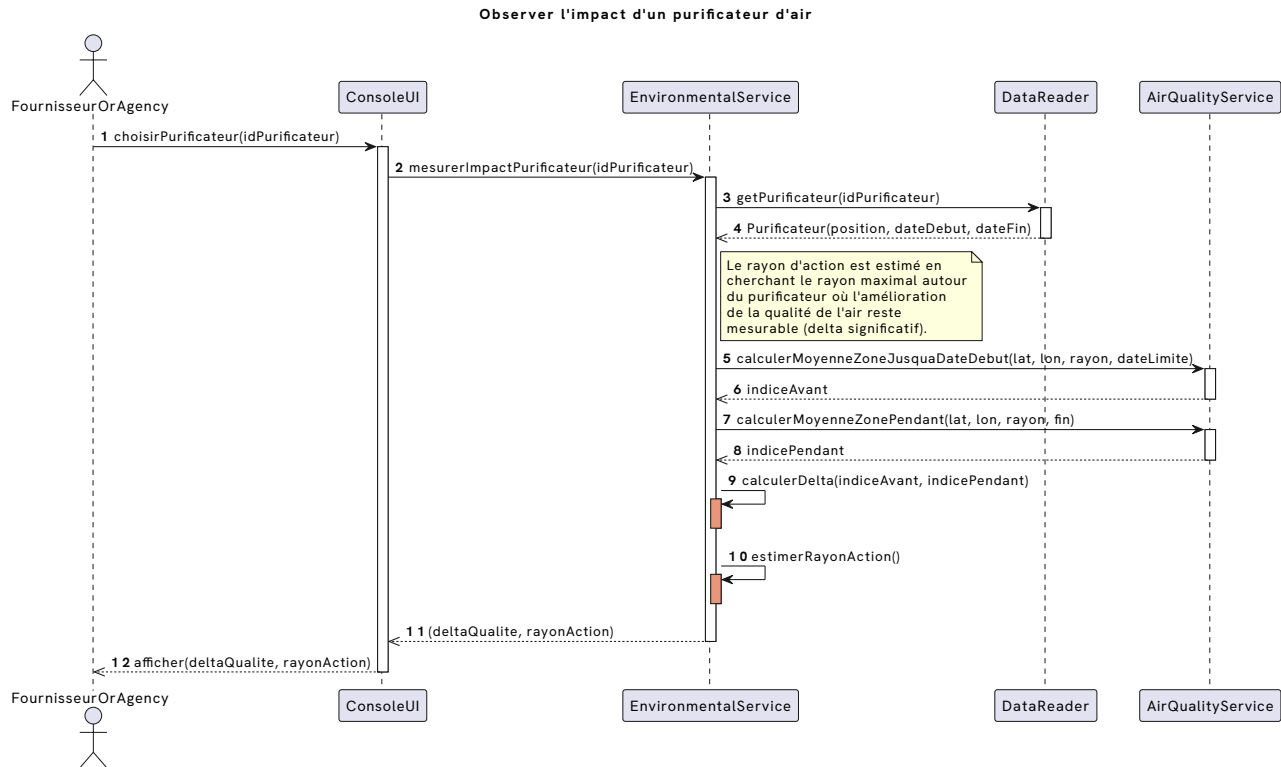


Fig. 6 : Scénario 5 — Observer l'impact d'un purificateur d'air

F. Scénario 6 — Classifier la fiabilité d'un capteur privé

Objectif : Évaluer la fiabilité du capteur d'un particulier en comparant ses données à celles des capteurs officiels voisins, et mettre à jour son statut dans le système.

Acteur : Agence.

Déroulement : L'agence sélectionne un particulier et son capteur. `AdministrationService.classifierFiabilite()` récupère les mesures du capteur privé et celles des capteurs officiels voisins. Pour chacun des quatre attributs (O3, SO2, NO2, PM10), trois critères d'anomalie sont vérifiés : (1) écart trop important par rapport à la moyenne du voisinage (supérieur à K fois l'écart-type), (2) valeurs constantes suspectes (variance proche de zéro), (3) valeurs hors bornes physiques de l'attribut. Si le nombre total d'anomalies est inférieur ou égal au seuil de tolérance, le capteur est marqué **FIABLE** et ses données sont conservées. Dans le cas contraire, il est marqué **NON FIABLE**, toutes ses mesures sont invalidées, et l'attribution de points au particulier est désactivée.

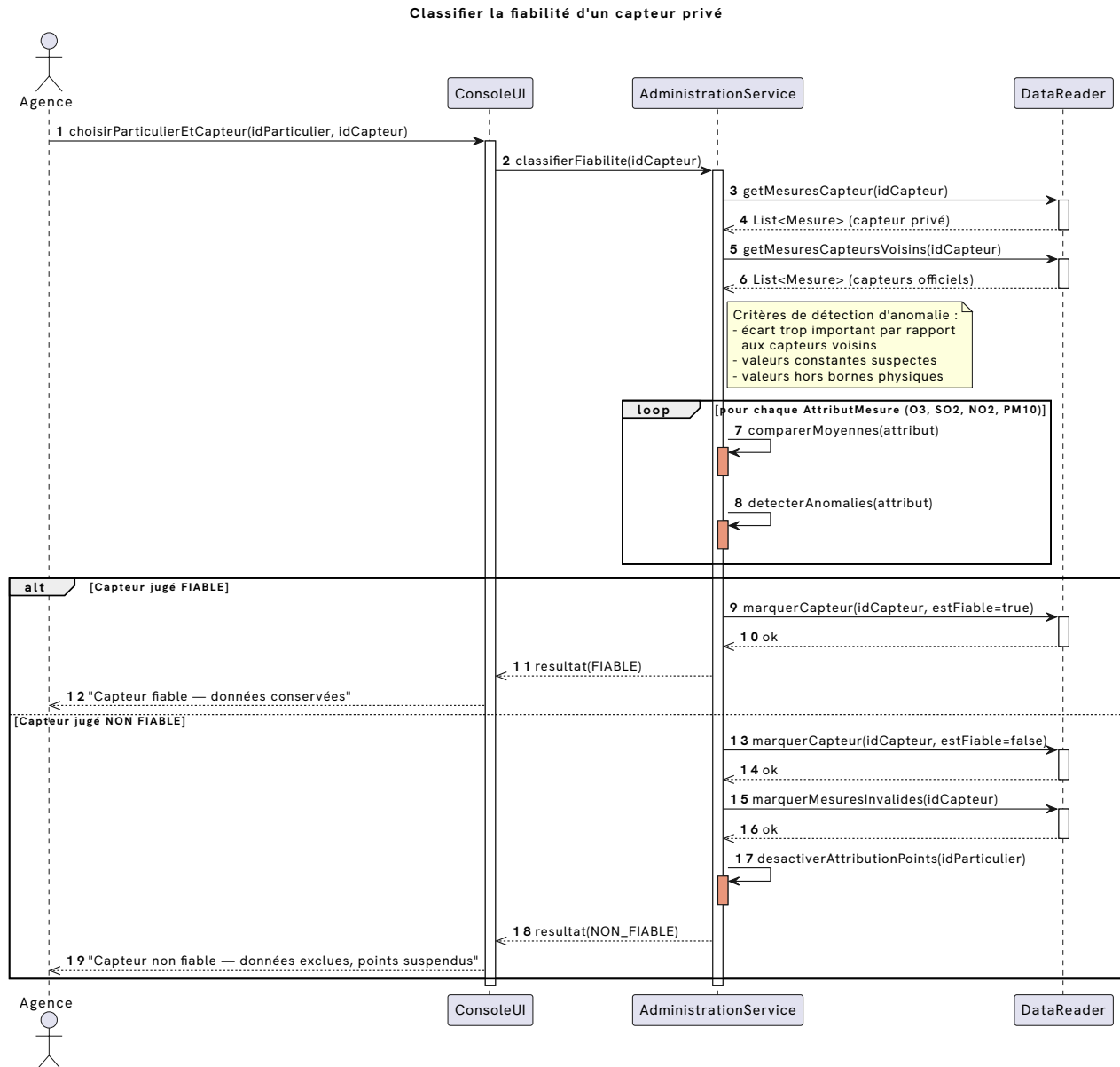


Fig. 7 : Scénario 6 — Classier la fiabilité d'un capteur privé

IV. Description et pseudo-code des algorithmes principaux

Les algorithmes présentés ci-dessous correspondent aux méthodes de service les plus critiques de l'application. Chaque algorithme est encadré par des appels à PerformanceMonitor afin de mesurer son temps d'exécution.

IV.1. Algorithme 1 — AdministrationService.analyserCapteur

Description : Analyse un capteur en comparant ses relevés à ceux de ses voisins pour détecter des anomalies (valeurs hors bornes, écart anormal, valeurs constantes suspectes).

Entrée : idCapteur : String

Sortie : rapport { etat ∈ {FONCTIONNEL, DÉFAILLANT, INTROUVABLE}, anomalies[] }

ALGORITHME AdministrationService.analyserCapteur

ENTRÉE : idCapteur : String

SORTIE : rapport { etat, anomalies[] }

DÉBUT

```
PerformanceMonitor.demarrer("analyserCapteur")
```

```
capteur ← DataReader.getCapteur(idCapteur)
```

```
SI capteur = NULL ALORS
```

```
    PerformanceMonitor.arreter("analyserCapteur")
```

```
    RETOURNER rapport(etat = INTROUVABLE)
```

```
FIN SI
```

```
mesures ← DataReader.getMesuresCapteur(idCapteur)
```

```
voisins ← DataReader.getCapteursVoisins(idCapteur)
```

```
mesuresVois ← DataReader.getMesuresCapteursVoisins(idCapteur)
```

```
anomalies ← liste vide
```

```
POUR CHAQUE attribut DANS {O3, SO2, NO2, PM10} FAIRE
```

```
    moyenneCapteur ← moyenne(mesures, attribut)
```

```
    moyenneVoisinage ← moyenne(mesuresVois, attribut)
```

```
    ecart ← |moyenneCapteur - moyenneVoisinage|
```

```
    SI moyenneCapteur HORS bornesPhysiques(attribut) ALORS
```

```
        anomalies.ajouter("Valeur hors bornes : " + attribut)
```

```
    SI ecart > SEUIL_ECART(attribut) ALORS
```

```
        anomalies.ajouter("Écart anormal vs voisins : " + attribut)
```

```
    SI varianceConstante(mesures, attribut) ALORS
```

```
        anomalies.ajouter("Valeurs constantes suspectes : " + attribut)
```

```
FIN POUR
```

```
SI anomalies est vide ALORS
```

```
    rapport ← (etat = FONCTIONNEL, anomalies = [])
```

```
SINON
```

```
    rapport ← (etat = DEFAILLANT, anomalies)
```

```
FIN SI
```

```
PerformanceMonitor.arreter("analyserCapteur")
```

```
RETOURNER rapport
```

FIN

IV.2. Algorithme 2 — AirQualityService.calculerMoyenneAQI

Description : Calcule l'indice ATMO moyen sur une zone circulaire pour une période donnée, en excluant les capteurs non fiables, et attribue des points aux particuliers contributeurs.

Entrées : lat, lon : double, rayon : double, debut, fin : DateTime

Sortie : indiceATMO : double

ALGORITHME AirQualityService.calculerMoyenneAQI

ENTRÉES : lat, lon, rayon, debut, fin

SORTIE : indiceATMO : double

DÉBUT

```
PerformanceMonitor.demarrer("calculerMoyenneAQI")

capteursZone      ← DataReader.getCapteursValidesDansRayon(lat, lon, rayon)
toutesMesures      ← liste vide
particuliersImpliques ← ensemble vide

POUR CHAQUE c DANS capteursZone FAIRE
    // Les capteurs non fiables sont déjà exclus par getCapteursValidesDansRayon()
    mesures ← DataReader.getMesuresPeriode(c.id, debut, fin)
    mesures ← filtrer(mesures, m → m.estValide = VRAI)
    SI mesures est vide ALORS CONTINUER FIN SI
    toutesMesures.ajouterTout(mesures)
    SI c.proprietaire est un Particulier ALORS
        particuliersImpliques.ajouter(c.proprietaire)
    FIN SI
FIN POUR

indiceATMO ← AirQualityService.calculerIndiceATMOMoyen(toutesMesures)
AdministrationService.attribuerPoints(particuliersImpliques)

PerformanceMonitor.arreter("calculerMoyenneAQI")
RETOURNER indiceATMO
FIN
```

IV.3. Algorithme 3 — AirQualityService.comparerCapteurs

Description : Classe tous les capteurs par similarité croissante par rapport à un capteur de référence sur une période, en utilisant la distance euclidienne entre les vecteurs de moyennes par attribut.

Entrées : idRef : String, debut : DateTime, fin : DateTime

Sortie : classement : List<Capteur> (trié par score croissant)

ALGORITHME AirQualityService.comparerCapteurs
ENTRÉES : idRef : String, debut : DateTime, fin : DateTime
SORTIE : classement : List<Capteur>

DÉBUT

```
PerformanceMonitor.demarrer("comparerCapteurs")

mesuresRef      ← DataReader.getMesuresPeriode(idRef, debut, fin)
autresCapteurs ← DataReader.getAutresCapteurs(idRef)
classement      ← liste vide

POUR CHAQUE c DANS autresCapteurs FAIRE
    mesuresC ← DataReader.getMesuresPeriode(c.id, debut, fin)
    SI mesuresC est vide ALORS CONTINUER FIN SI

    // Distance euclidienne entre vecteurs de moyennes (O3, SO2, NO2, PM10)
    // Plus le score est bas, plus le capteur est similaire à la référence
    score ← AirQualityService.calculerScoreSimilarite(mesuresRef, mesuresC)
    classement.ajouter((c, score))
```

FIN POUR

```
AirQualityService.trierParScoreCroissant(classement)
```

```
PerformanceMonitor.arreter("comparerCapteurs")
```

RETOURNER classement

FIN

IV.4. Algorithme 4 — AirQualityService.estimerQualitePosition

Description : Estime l'indice ATMO à une position précise et un instant donné. Si un capteur est présent, ses mesures sont utilisées directement. Sinon, une interpolation spatiale pondérée par l'inverse de la distance (IDW) est effectuée.

Entrées : lat, lon : double, timestamp : DateTime

Sortie : indiceATMO : double

ALGORITHME AirQualityService.estimerQualitePosition

ENTRÉES : lat, lon, timestamp

SORTIE : indiceATMO : double

DÉBUT

```
PerformanceMonitor.demarrer("estimerQualitePosition")
```

```
capteursAPosition ← DataReader.getTousCapteursLatLon(lat, lon)
```

```
particuliersImpliques ← ensemble vide
```

SI capteursAPosition n'est pas vide ALORS

```
capteurExact ← capteursAPosition[0]
```

```
mesures ← DataReader.getMesuresAuTimestamp(capteurExact.id, timestamp)
```

```
indiceATMO ← AirQualityService.calculerIndiceATMO(mesures)
```

SI capteurExact.proprietaire est un Particulier ALORS

```
particuliersImpliques.ajouter(capteurExact.proprietaire)
```

FIN SI

SINON

```
// Interpolation spatiale pondérée par l'inverse de la distance
```

```
tousCapteurs ← DataReader.getTousCapteurs()
```

```
proches ← N capteurs les plus proches de (lat, lon) dans tousCapteurs
```

POUR CHAQUE c DANS proches FAIRE

```
mesuresC ← DataReader.getMesuresAuTimestamp(c.id, timestamp)
```

```
attacher mesuresC à c
```

SI c.proprietaire est un Particulier ALORS

```
particuliersImpliques.ajouter(c.proprietaire)
```

FIN SI

FIN POUR

```
indiceATMO ← AirQualityService.interpolerPondereeParDistance(  
    proches, lat, lon)
```

FIN SI

```
AdministrationService.attribuerPoints(particuliersImpliques)
```

```
PerformanceMonitor.arreter("estimerQualitePosition")
```

```
RETOURNER indiceATMO
```

```
FIN
```

IV.5. Algorithme 5 — EnvironmentalService.mesurerImpactPurificateur

Description : Mesure l'impact d'un purificateur en calculant le delta d'indice ATMO avant et pendant son fonctionnement, et en estimant son rayon d'action effectif via une recherche par incréments.

Entrée : idPurificateur : String

Sortie : ImpactPurificateur { delta : double, rayonAction : double }

ALGORITHME EnvironmentalService.mesurerImpactPurificateur

ENTRÉE : idPurificateur : String

SORTIE : ImpactPurificateur { delta, rayonAction }

DÉBUT

```
PerformanceMonitor.demarrer("mesurerImpactPurificateur")
```

```
purif ← DataReader.getPurificateur(idPurificateur)
```

```
SI purif = NULL ALORS
```

```
PerformanceMonitor.arreter("mesurerImpactPurificateur")
```

```
RETOURNER ÉCHEC
```

```
FIN SI
```

```
rayonAction ← EnvironmentalService.estimerRayonAction(purif)
```

```
indiceAvant ← AirQualityService.calculerMoyenneZoneJusquaDateDebut(  
    purif.latitude, purif.longitude, rayonAction, purif.dateDebut)
```

```
indicePendant ← AirQualityService.calculerMoyenneZonePendant(  
    purif.latitude, purif.longitude, rayonAction,  
    purif.dateDebut, purif.dateFin)
```

```
delta ← EnvironmentalService.calculerDelta(indiceAvant, indicePendant)
```

```
PerformanceMonitor.arreter("mesurerImpactPurificateur")
```

```
RETOURNER ImpactPurificateur(delta, rayonAction)
```

```
FIN
```

SOUS-ALGORITHME EnvironmentalService.estimerRayonAction

ENTRÉE : purif : Purificateur

SORTIE : rayon : double

DÉBUT

```
rayonRetenu ← 0
```

```
POUR r DE RAYON_MIN À RAYON_MAX PAR PAS pas FAIRE
```

```
avant ← AirQualityService.calculerMoyenneZoneJusquaDateDebut(  
    purif.latitude, purif.longitude, r, purif.dateDebut)
```

```
pendant ← AirQualityService.calculerMoyenneZonePendant(  
    purif.latitude, purif.longitude, r,
```

```
                purif.dateDebut, purif.dateFin)
SI (avant – pendant) >= SEUIL_AMELIORATION_SIGNIFICATIVE ALORS
    rayonRetenu ← r
SINON
    SORTIR DE LA BOUCLE
FIN SI
FIN POUR
RETOURNER rayonRetenu
FIN
```

IV.6. Algorithme 6 — AdministrationService.classifierFiabilite

Description : Évalue la fiabilité d'un capteur privé en détectant les anomalies sur chacun des quatre attributs de mesure et met à jour le statut du capteur et de son propriétaire dans le système.

Entrée : idCapteur : String

Sortie : estFiable : bool

ALGORITHME AdministrationService.classifierFiabilite

ENTRÉE : idCapteur : String

SORTIE : estFiable : bool

DÉBUT

```
PerformanceMonitor.demarrer("classifierFiabilite")
```

```
capteur ← DataReader.getCapteur(idCapteur)
```

```
particulier ← capteur.proprietaire
```

```
mesuresPrive ← DataReader.getMesuresCapteur(idCapteur)
```

```
mesuresVoisins ← DataReader.getMesuresCapteursVoisins(idCapteur)
```

```
nbAnomalies ← 0
```

```
POUR CHAQUE attribut DANS {O3, SO2, NO2, PM10} FAIRE
```

```
    moyennePrive ← moyenne(mesuresPrive, attribut)
```

```
    moyenneVoisinage ← moyenne(mesuresVoisins, attribut)
```

```
    ecartType ← ecartType(mesuresVoisins, attribut)
```

```
// Critère 1 : écart trop important par rapport aux capteurs voisins
```

```
SI |moyennePrive – moyenneVoisinage| > K × ecartType ALORS
```

```
    nbAnomalies ← nbAnomalies + 1
```

```
// Critère 2 : valeurs constantes suspectes (variance ≈ 0)
```

```
SI varianceConstante(mesuresPrive, attribut) ALORS
```

```
    nbAnomalies ← nbAnomalies + 1
```

```
// Critère 3 : valeurs hors bornes physiques
```

```
SI moyennePrive HORS bornesPhysiques(attribut) ALORS
```

```
    nbAnomalies ← nbAnomalies + 1
```

```
FIN POUR
```

```
SI nbAnomalies <= SEUIL_TOLERANCE ALORS
```

```
    capteur.estFiable ← VRAI
```

```
    particulier.estFiable ← VRAI
    DataReader.marquerCapteur(idCapteur, VRAI)
    DataReader.sauvegarderCapteur(capteur)
    estFiable ← VRAI
SINON
    capteur.estFiable      ← FAUX
    particulier.estFiable ← FAUX
    DataReader.marquerCapteur(idCapteur, FAUX)
    DataReader.marquerMesuresInvalides(idCapteur)
    DataReader.sauvegarderCapteur(capteur)
    AdministrationService.desactiverAttributionPoints(particulier.id)
    estFiable ← FAUX
FIN SI

PerformanceMonitor.arreter("classifierFiabilite")
RETOURNER estFiable
FIN
```